



Apache Kafka — Day 2

Producer & Consumer APIs, Topic & Partition Management

8 HOURS

INTERMEDIATE

CONCEPT → CODE → CONFIG → DEMO → HANDS-ON

SECTION 1

Producer & Consumer API

4 Hours

Day 1 Recap

Broker · Topic · Partition · Producer · Consumer · Offset
· Consumer Group · Replication · Leader/Follower · ISR

Day 2 Goal

Build applications that actually interact with Kafka using the Producer and Consumer APIs.

Day 2 Learning Objectives

→ Producers

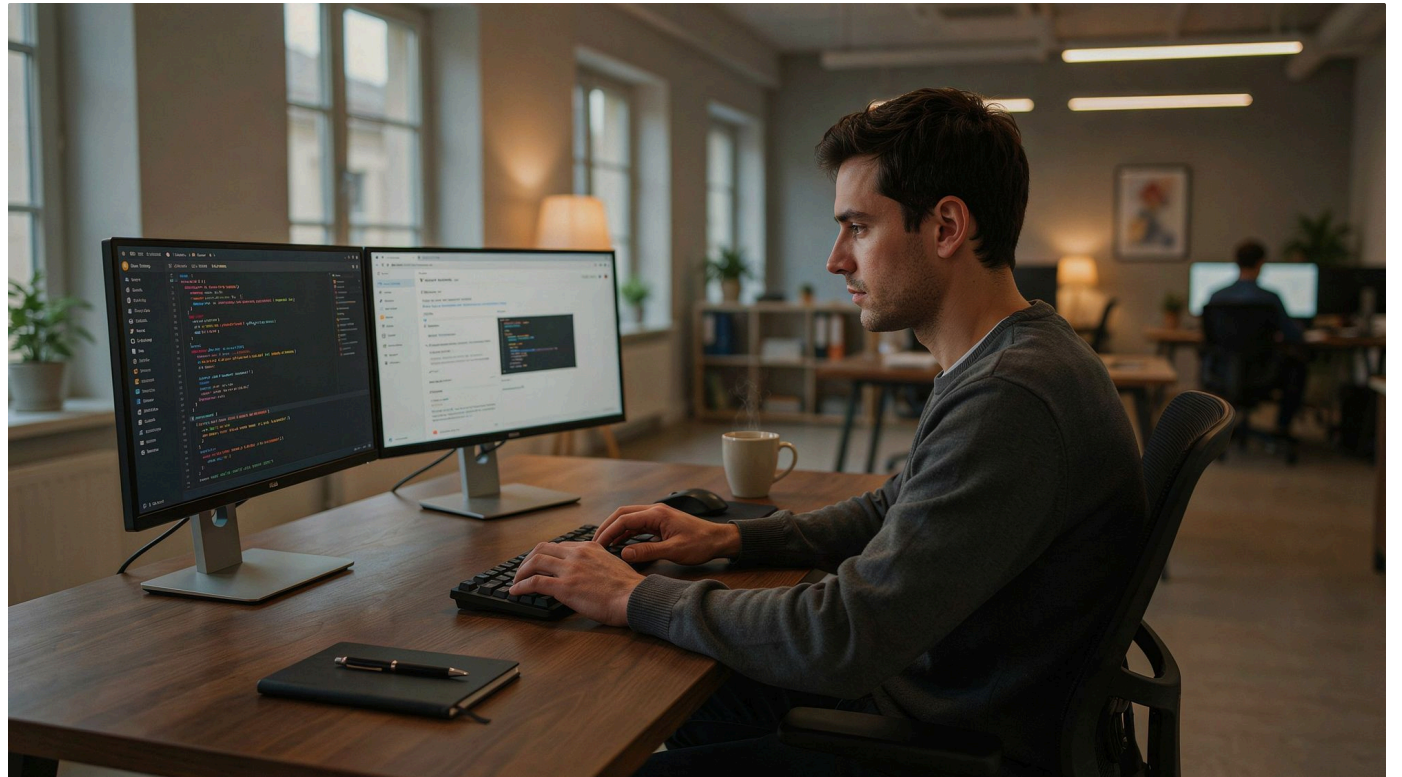
Build producers, understand sync vs async, custom partitioning, serializers

→ Consumers

Build consumers, scale with consumer groups, handle rebalancing, manage lag

→ Operations

Manage topics & partitions, configure retention & compression, explore DR



Kafka Client APIs

Applications interact with Kafka via **Kafka Client APIs** — not CLI commands.

Producer API Application → Kafka Send records to topics	Kafka Cluster B1 · B2 · B3 Brokers store and route data	Consumer API Kafka → Application Read records from topics
--	--	--

This course applies these concepts to **Java/.NET producers and consumers**.



Kafka Producer API

A producer creates and publishes records. A `ProducerRecord` requires: **topic**, **key**, **value** — plus broker address, serializer, and producer configuration.

Bootstrap Address

Kafka broker connection

Topic + Key + Value

Record destination and content

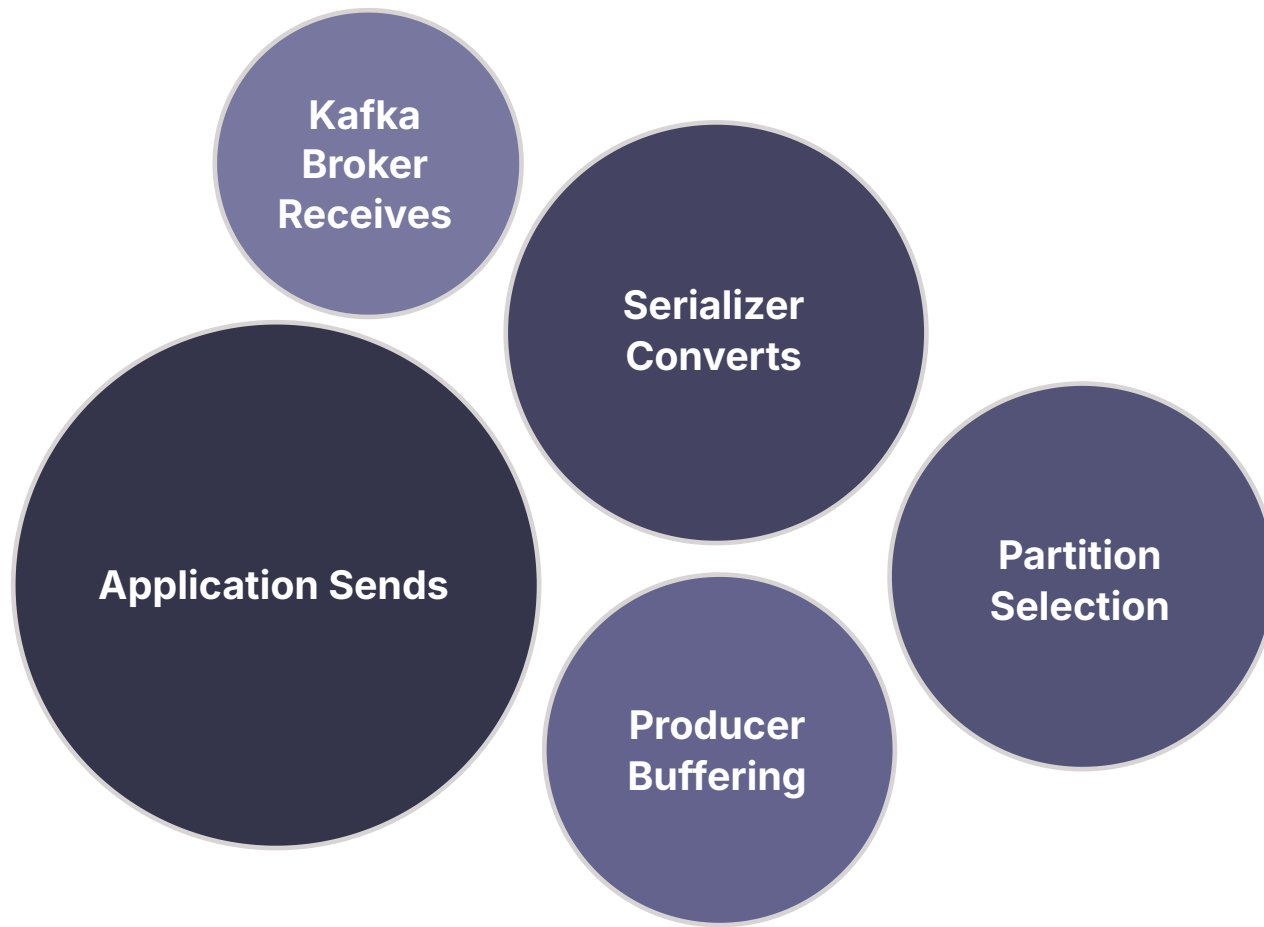
Serializer

Converts objects to bytes

Configuration

acks, retries, batching, etc.

Producer Message Flow



Key Configurations Along This Flow

batch.size / linger.ms

Control batching behavior

buffer.memory

Producer buffer capacity

compression.type

Payload compression

acks / retries

Reliability guarantees

Synchronous vs Asynchronous Producer

Synchronous

Application waits for acknowledgement before continuing.

- ✅ Straightforward error handling
- ⚠️ Lower throughput — blocks per record

Use when confirmation of acceptance is required before proceeding.

Asynchronous

Application fires records without blocking; results handled via callbacks.

- ✅ Better throughput & resource utilization
- ⚠️ Requires async error & retry handling

Natural fit for batching and high-volume workloads.

📝 Discussion: Should an IoT platform processing 100,000 sensor events/sec block on every individual send?



Why Kafka Needs Serialization

Applications work with objects. Kafka transmits and stores **bytes**. Serialization bridges the two.

1

Application Object

e.g. Order { id, customerId, amount }

2

Serializer

Converts object → bytes

3

Kafka

Stores & transmits bytes

4

Deserializer

Bytes → Object on consumer side

Custom Serializer

When Standard Formats Aren't Enough

Standard options include String, JSON, and binary formats. A **custom serializer** converts domain objects into a consistent Kafka message representation.

Example: `PaymentTransaction` → `PaymentSerializer` →
Kafka Record

Organizations use custom serializers to enforce consistent message structure across producers.

String

Simple text payloads

JSON

Human-readable structured data

Binary

Compact, schema-driven formats

Custom

Domain-specific serialization logic

Partitioning: How a Producer Chooses a Partition

Each producer record must be routed to a partition. The **record key** is central to this decision.

1 No Key

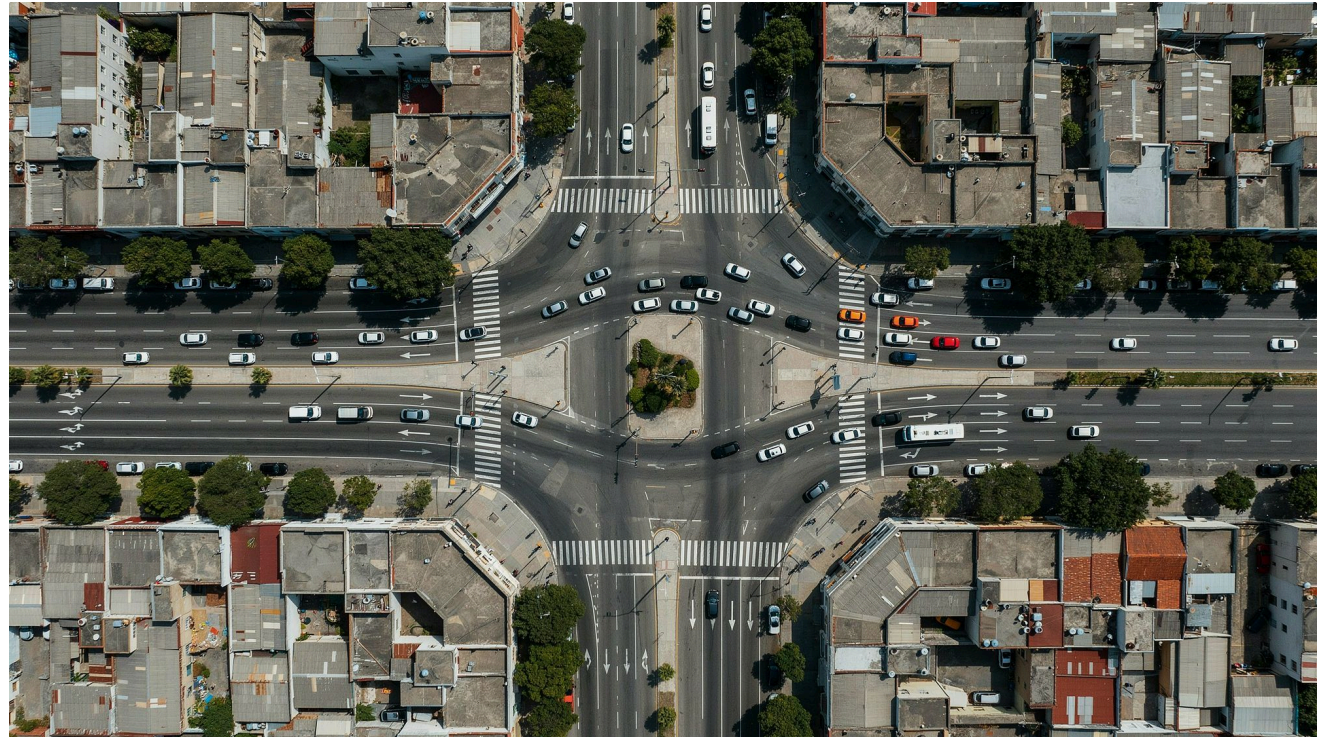
Round-robin or sticky partitioning across available partitions

2 With Key

Hash of key deterministically maps to a partition — same key always lands on same partition

3 Custom Partitioner

Application-defined logic overrides default behavior



Custom Partitioner

Default partitioning isn't always sufficient. A custom partitioner routes records based on business logic.

Example Requirement

Priority Customers → **Partition 0**
Regular Customers → **Partitions 1–3**

Implementation

Implement the `Partitioner` interface and apply business rules to determine the target partition.

Hot Partitions

Poor distribution concentrates load on one partition. Design custom partitioners carefully to ensure even spread.



Kafka Consumer API

Consumer applications read records from Kafka topics and drive business logic.

01

Connect & Subscribe

Connect to Kafka and subscribe to one or more topics

02

Poll & Deserialize

Poll for records and deserialize bytes back into application objects

03

Process

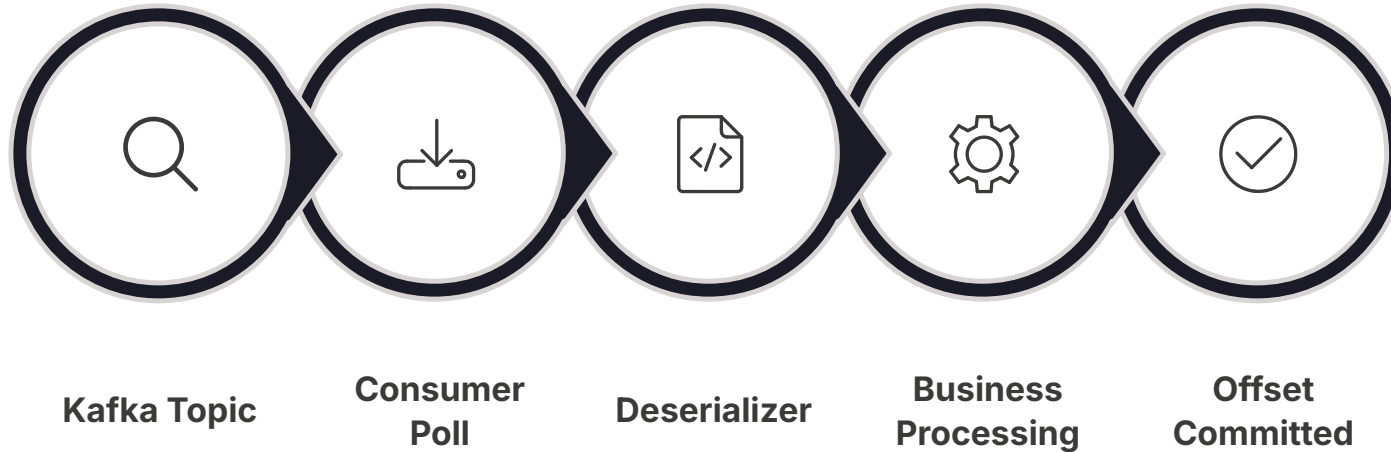
Execute business logic — validate, enrich, forward to downstream systems

04

Commit Offset

Track consumption progress to enable reliable replay and recovery

Consumer Processing Flow



Reliability Depends on More Than Kafka

Consumer reliability is shaped by:

- Application processing correctness
- Offset management strategy (auto vs manual commit)
- Downstream system availability
- Error and retry handling

Single Consumer → Scaling Consumers

Single Consumer

P0, P1, P2 → **Consumer 1**

Works well for low traffic and lightweight processing.
Becomes a bottleneck as volume grows.

Scaled Consumer Group

P0 → C1 · P1 → C2 · P2 → C3

All in `order-processing` group. **Partitions determine maximum useful parallelism.** Extra consumers beyond partition count sit idle.

- ❏ With 3 partitions and 5 consumers, 2 consumers will receive no partition assignment.

Multithreaded Consumers

Two Scaling Dimensions Design Considerations

- Message ordering guarantees
- Thread safety
- Offset management & processing completion
- Error handling per thread



Scale Out

Multiple consumer instances in the same consumer group

Internal Threading

Single consumer + worker thread pool for parallel processing



Consumer Group Rebalancing

When group membership or partition topology changes, Kafka redistributes partition assignments — this is **rebalancing**.

1

Consumer Joins

New instance added to group → rebalance triggered

2

Consumer Leaves / Fails

Instance removed → partitions redistributed to remaining consumers

3

Partition Changes

Topic partition count changes → assignments may shift

During rebalancing, processing behavior can be affected. Applications must handle partition ownership, offset state, and lifecycle events correctly.

Hands-on Lab 1: Producer & Consumer APIs

1

Build Producer & Consumer

Configure and run a text producer and consumer against the `orders` topic

2

Sync vs Async

Compare synchronous and asynchronous producer behavior; add message keys and observe partition assignment

3

Custom Serializer

Introduce a domain-object serializer and deserializer

4

Custom Partitioner

Implement routing logic; run multiple consumers in one group and observe distribution

✓ Outcome: Participants move from Kafka CLI to application-level Kafka integration.

SECTION 2

Topic & Partition Management

4 Hours

Kafka is not just *create topic* → *send messages*. Operational behavior depends on deliberate configuration choices.

Performance

Partitions, batching, compression

Availability

Replication, acks, retries

Storage

Retention, segment sizing, compaction

Scalability

Consumer lag, partition distribution, reassignment

Compacted Topics

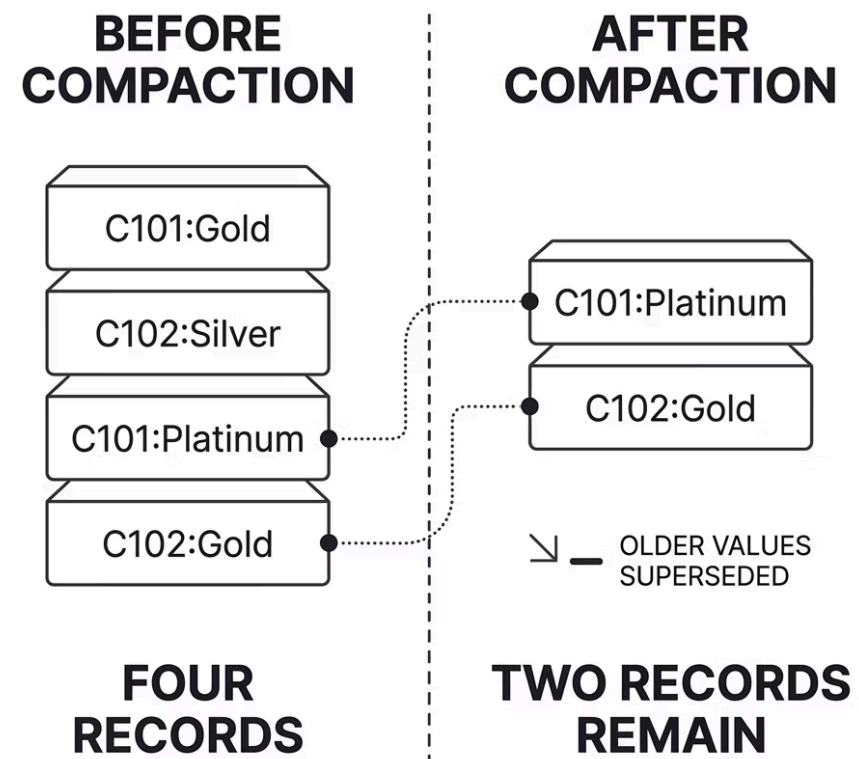
Retain the Latest Value per Key

For some use cases, only the **most recent value for each key** matters. Compaction allows Kafka to remove superseded records over time.

Example: C101 → Gold → Platinum — only **Platinum** is retained.

Useful For

- Customer / account state
- Configuration & reference data
- Materialized-state reconstruction



Kafka Compression



Reduce Network & Storage Overhead

Compression is applied at the **batch level**. A batch of events is compressed before transmission to the broker.

Key config: `compression.type` — options include `none`, `gzip`, `snappy`, `lz4`, `zstd`.

Benefits

Reduced network usage · Lower storage requirements

Trade-off

Additional CPU overhead for compression and decompression

Critical Producer Configurations

acks

Acknowledgement expectations — trades reliability for latency

buffer.memory

Memory available for producer buffering before blocking

retries

Retry behavior on transient failures

batch.size / linger.ms

Batch capacity and wait time to form larger batches

max.in.flight.requests

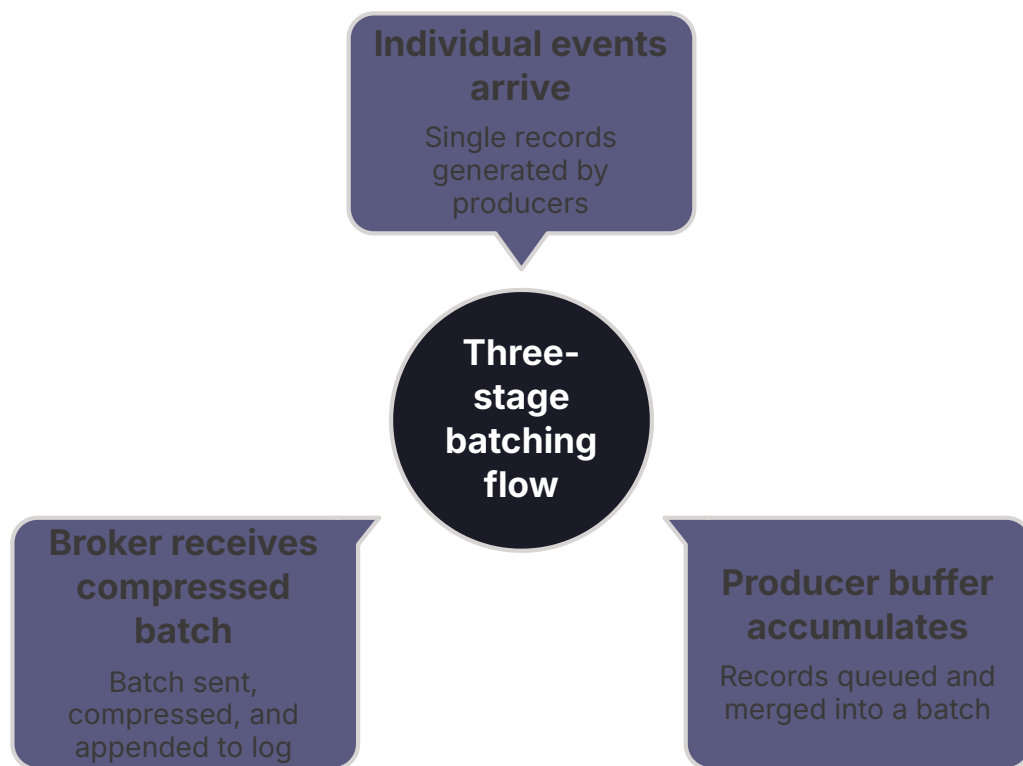
Outstanding requests per connection — affects ordering

client.id

Identifies the producer client for monitoring

❏ These settings trade off: Throughput · Latency · Reliability · Memory · Network usage

Batching: `batch.size` + `linger.ms`



Why Batch?

Sending every record individually is inefficient. Batching amortizes network overhead across multiple records.

Trade-off: More batching → better throughput, but potential additional latency.

`batch.size`

Maximum bytes per batch — larger batches = fewer requests

`linger.ms`

Wait time before sending an underfilled batch — allows more records to accumulate

Topic, Partition & Retention Configuration

num.partitions

Default partition count for new topics

log.retention.ms

Time-based retention — how long events are kept

log.retention.bytes

Size-based retention limit per partition

log.segment.bytes

Controls log segment file sizing

log.segment.ms

Time-based segment rolling behavior

message.max.bytes

Maximum message size at broker/topic level

Kafka Retention



Retention Is an Architecture Decision

Kafka does not keep events forever by default. Retention is driven by time (`log.retention.ms`) and/or size limits.

Questions to Answer

- What replay window is required?
- What are downstream recovery needs?
- What storage capacity is available?
- Are there regulatory or business requirements?

Example: How long should financial transaction events remain in Kafka?

Consumer Lag & Backpressure

When consumers process slower than producers publish, **lag accumulates**.

1,000

Events/sec

Producer rate

600

Events/sec

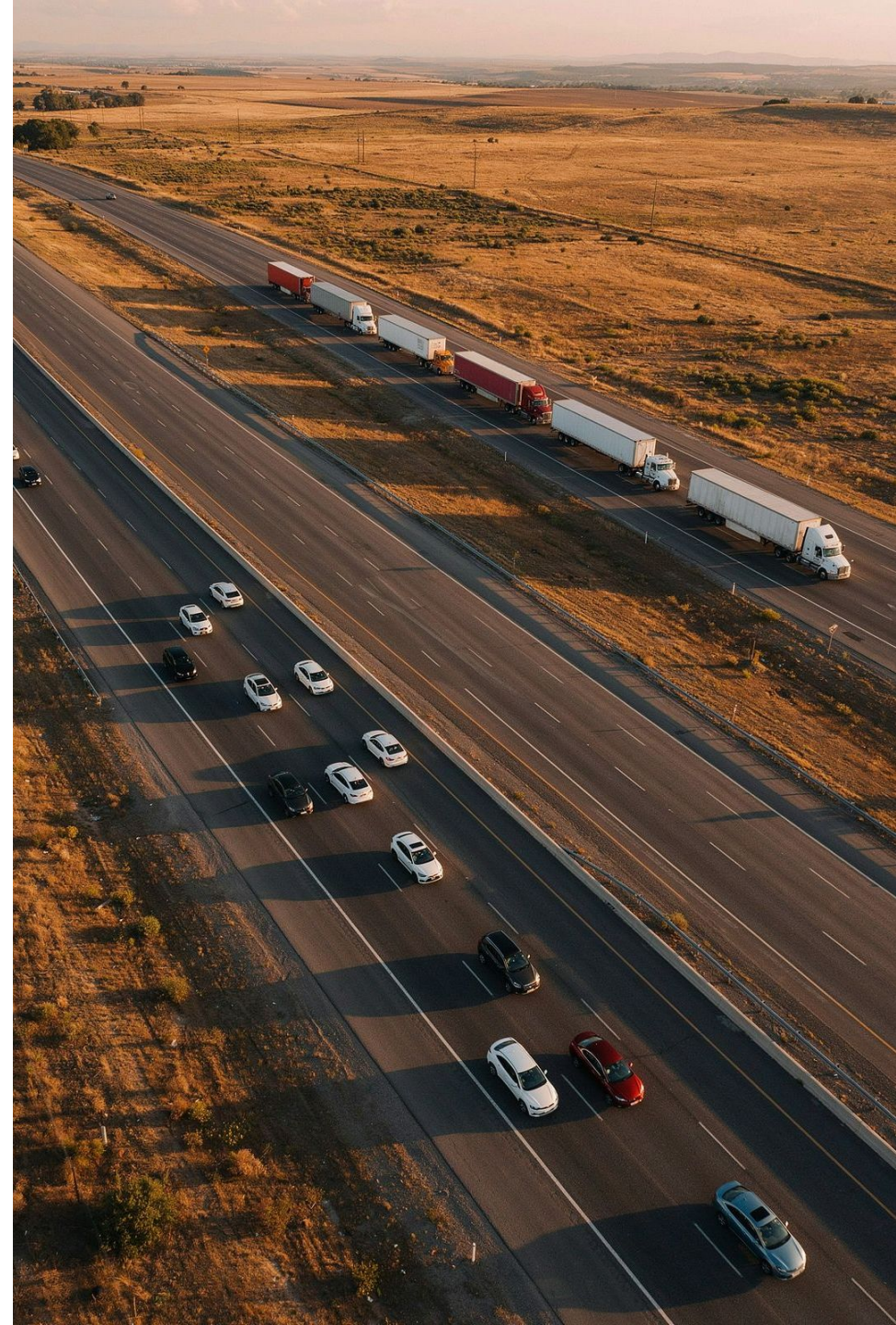
Consumer rate

400

Events/sec

Accumulating lag

Consumer lag measures how far a consumer group is behind the latest available offset. Large or **increasing** lag signals that consumption is not keeping pace.



What Can Cause Consumer Lag?

Processing

Slow business logic · Processing failures & retries · Large messages

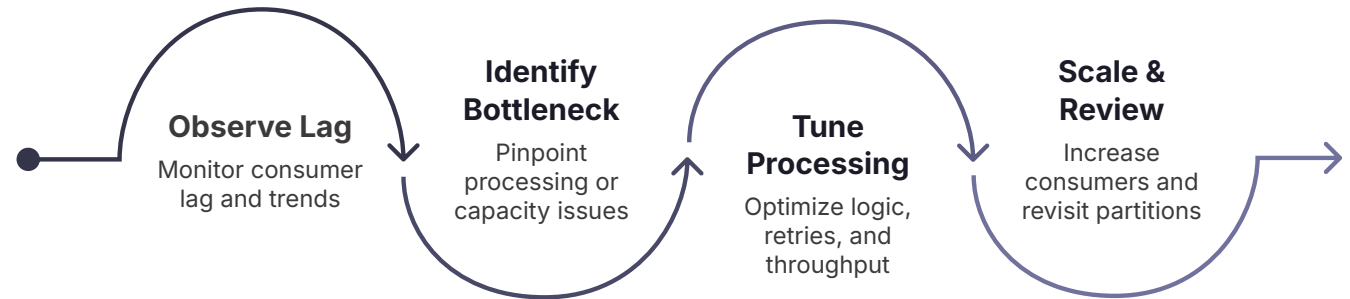
Capacity

Insufficient consumers · Too few partitions for desired parallelism

External

Downstream DB/API bottleneck · Network constraints · Traffic spikes

Typical Response



Do not treat "add consumers" as the answer to every lag problem.

Deleting Records and Topics



Administrative Operations

Kafka supports deleting specific records, all records from a topic, or an entire topic. These operations can have significant consequences.

Before Deleting, Understand

- Which applications produce/consume this topic
- Active consumer groups and their offsets
- Downstream dependencies

⚠ Use a disposable training topic for demonstrations — not a shared production topic.

Kafka Log Directories

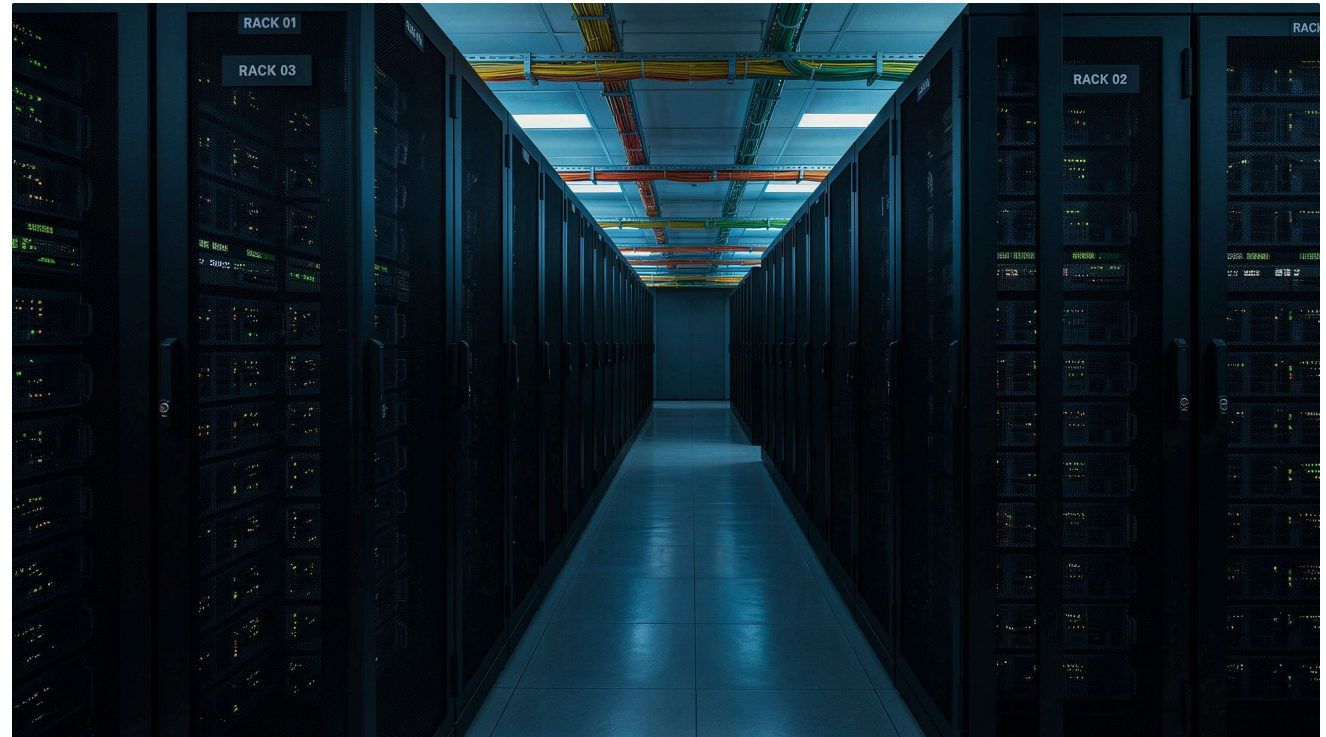
How Brokers Persist Data

Each broker stores partition data in configured **log directories**. Understanding this layout helps administrators reason about storage and disk utilization.

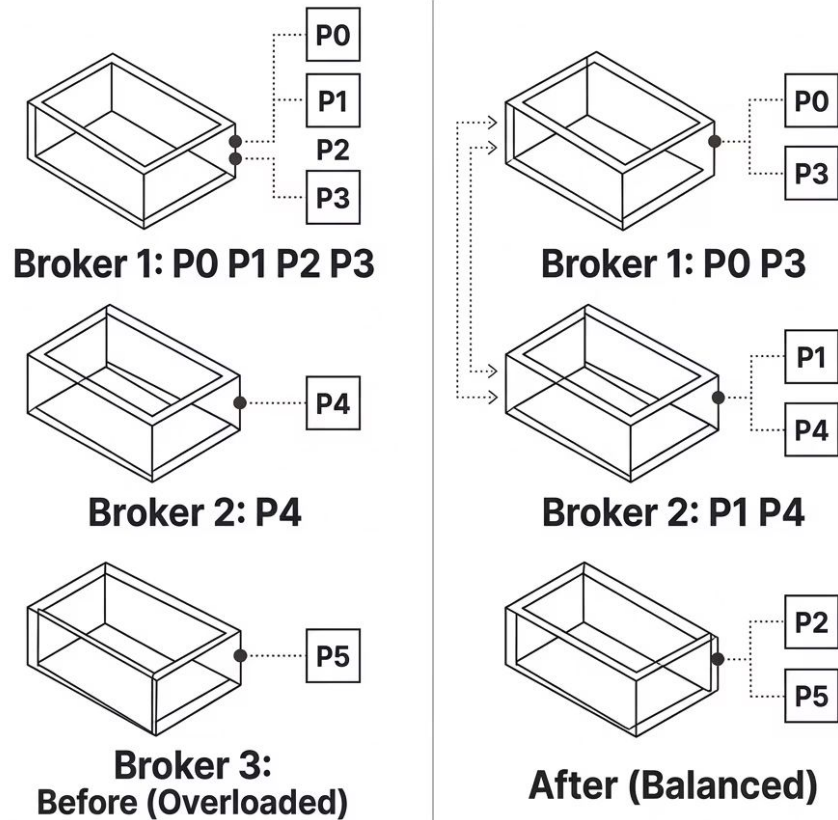
Broker

- └─ /kafka-logs/
 - └─ orders-0/
 - └─ orders-1/
 - └─ payments-0/
 - └─ payments-1/

Inspected via `kafka-log-dirs.sh` — useful for capacity planning and diagnosing storage imbalances.



Partition Reassignment



Why Reassign Partitions?

→ Add / Remove Brokers

Rebalance load across new or remaining brokers

→ Improve Distribution

Correct uneven partition placement

→ Maintenance

Move replicas before decommissioning a broker



Disaster Recovery & MirrorMaker

MirrorMaker replicates Kafka data between clusters — enabling DR, cross-site data movement, and cluster migration.

Primary Cluster

Region / DC A — active production cluster

MirrorMaker

Replicates topics and offsets between clusters

Secondary Cluster

Region / DC B — standby or active-active target



Day 2 introduces DR concepts. Deeper DR design patterns are covered in later sessions.

Hands-on Lab 2: Compaction & Topic Configuration

1

Compacted Topic

Write multiple updates per key; observe how compaction retains only the latest value per key

2

Topic Configuration

Inspect and modify retention, segment, and message-size settings

3

Retention

Explore time-based and size-based retention behavior

4

Consumer Lag

Run producer faster than consumer; observe lag accumulating in real time

Hands-on Lab 3: Partition & Broker Operations

Partition Inspection

Inspect partitions, leaders, replicas, and ISR state

Partition Reassignment

Move partition replicas between brokers and verify new distribution

Log Directories

Inspect broker log-directory information and storage layout

Multi-Broker Overrides

Explore configuration overrides in the training multi-broker environment

Docker Demo

Demonstrate broker operations using the Docker-based training cluster



Enterprise Case Study

High-Volume Payment Processing

20K

Transactions/sec

Platform throughput requirement

3

Consumer Groups

Fraud · Analytics · Settlement

Architecture: Payment Gateway → Kafka Producer → payment-
transactions (P0-P5+) → Consumer Groups

Case Study: Producer Decisions

1

Sync or Async?

At 20K TPS, async with batching is essential. Sync per-record would be a bottleneck.

2

What Key?

Use `accountId` or `transactionId` to ensure related events land on the same partition.

3

Compression?

Yes — payment records compress well. Reduces network and storage costs at scale.

4

Batching Strategy?

Tune `batch.size` and `linger.ms` to maximize throughput without unacceptable latency.

Case Study: Consumer & Operations Decisions

Consumer

How Many Consumers?

Match consumer count to partition count per group for full parallelism

Lag Detection

Monitor consumer group lag continuously; alert on sustained increase

Partition Scaling

Partition count sets the ceiling — plan ahead for growth

Operations

Retention

Define replay window based on fraud investigation and settlement SLAs

Adding Brokers

Partition reassignment required to distribute load to new brokers

Reassignment Triggers

Broker addition/removal, capacity changes, infrastructure maintenance

Day 2 — Key Takeaways

Producer API

Sync vs async, serialization, custom partitioning, and batching all shape throughput and reliability.

Consumer API

Consumer groups, rebalancing, and offset management are the foundation of scalable, reliable consumption.

Topic Management

Retention, compaction, compression, and partition configuration are deliberate architecture decisions.

Operations

Log directories, partition reassignment, consumer lag monitoring, and DR are essential operational skills.

